Activity: Sobel and Canny Edge Detection

Introduction

In this activity, you'll implement and compare two fundamental edge detection algorithms: **Sobel** and **Canny**, using Python. Edge detection is a critical technique in computer vision for identifying boundaries in images, such as object outlines. You'll apply these algorithms to sample images (`image1.jpg` and `image2.jpg` located in the `assets` folder), analyze the impact of parameters like kernel size (for Sobel) and thresholds (for Canny), and visualize the results to understand their strengths and limitations.

Learning Outcomes

By the end of this activity, you will:

1. Implement **Sobel** and **Canny** edge detection using Python and OpenCV.
2. Compare the performance of Sobel and Canny on different images.
3. Analyze the effect of **kernel size** (Sobel) and **threshold values** (Canny) on edge detection results.

Working with Dev Container

To complete this assignment in a reliable and fully configured environment, please refer to the instructions in the file: `README-devcontainer.md`. This guide walks you through opening the assignment in **Visual Studio Code** using a **Dev Container**, which automatically installs all necessary Python libraries and tools. Following that setup ensures that the notebook runs smoothly without manual configuration or missing dependencies. Make sure to open the **main activity folder** in VS Code and follow the steps outlined in the Dev Container guide before starting the notebook.

Starter File

- Open the notebook: `activity_sobel_canny_edge_detection.ipynb` in the Code folder.
- You can run it locally or in Google Colab for convenience.
- Ensure `scikit-image`, `opencv-python`, `matplotlib`, and `numpy` are installed.
- Sample images (`image1.jpg` and `image2.webp`) are located in the `assets` folder of the activity.

Activity Code Steps

This is a guided learning activity. You'll:

1. Load and preprocess the sample images in grayscale.
2. Implement Sobel edge detection with varying kernel sizes.
3. Implement Canny edge detection with adjustable threshold values.
4. Compare edge detection results and analyze parameter impacts.

Conclusion

In this activity, you successfully:

- Loaded and preprocessed sample images for edge detection.
- Implemented **Sobel** and **Canny** edge detection algorithms.
- Analyzed the effects of **kernel size** and **thresholds** on edge detection quality.
- Compared the performance of Sobel and Canny, noting Sobel's speed for approximations and Canny's precision for detailed edge maps.

You've gained practical experience with edge detection in computer vision.